Slide 1: MPI Matrix Multiplication

Speaker: Ehtesham Faraz

The Problem

Compute **C = A × B** on a distributed memory system where N×N matrices may be too large to fit in a single node's RAM.

N × N **A** × N × N **B** = N × N **C**

1D Row-Wise Decomposition

SPMD Approach

Single Program, Multiple Data. Each rank executes the same multiplication logic on its assigned row slice.

Specifics

Each rank holds `N/P` rows of A & C. Ranks `0..N%P-1` get one extra row to handle non-divisible N.

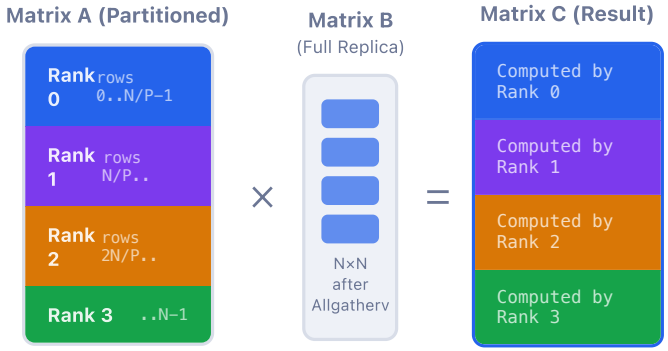
Load Balancing

Remainder rows distributed to lowest-ranked processes. All ranks differ by at most one row — near-perfect balance.

Partitioning Visualization

Ranks: P=4

Matrix A is split row-wise across ranks. Each rank generates its own slice of B, then **MPI_Allgatherv** replicates the full B. Result C is gathered to Rank 0.



mpi_matmul.c — row distribution

```
int base_rows = n / size;
int remainder = n % size;
int my_rows   = base_rows + (rank < remainder ? 1 : 0);

int my_offset = (rank < remainder)
    ? rank * (base_rows + 1)
    : remainder * (base_rows + 1) + (rank - remainder) * base_rows;
```